

Rappels sur les pointeurs, application au développement de *MyGPIO.c / .h*

1. Les variables ...

Une **variable** possède une **adresse**, une **valeur**.

Exemple :

`int a;` // la variable `a` est créée : on dit qu'elle est **définie**. A ce stade, elle possède donc une **adresse** (que l'on ne connaît pas explicitement, par exemple `0x2000 0000`). La valeur est indéfinie mais sa taille vaut 32bits (puisque `a` est un `int`).

`a=4;` // la valeur de la variable `a` vaut 4.

2. Les pointeurs ...

2.1. Explications, définitions

Lorsqu'on passe une variable via un paramètre à une fonction, cela peut se faire :

- par **valeur** : le paramètre de la fonction contient une **valeur**

exemple :

`void fct(int param);`

...

`a=4;`

`fct(a);`

`fct(6);`

- par **référence** (comprendre par **adresse**) : le paramètre de la fonction contient une adresse.

Exemple :

`int a;`

`void fct(int* param);`

`a=4;`

`fct(&a);` // NB & veut dire « adresse de la variable ... »

Le paramètre de la fonction *fct*, n'est plus de type *int*, mais *int **. Autrement dit, *param* est destiné à recevoir une adresse et rien d'autre !

param est un **pointeur**.

L'essentiel :

un **pointeur** est une **variable** mais **dont la valeur sera toujours une adresse**, donc de taille 32 bits, quel que soit l'objet pointé (ici un *int*).

Autre exemple :

char a; // la variable *a* est réservée, elle possède une adresse, mais aucune valeur encore.

char ptr;* // la variable *ptr* est réservée, de taille 32 bits mais non initialisé, *ptr* n'a pas encore de valeur... mais elle a évidemment une adresse.

void fct(char param);*

ptr = &a; // *ptr* est initialisé, en tant que pointeur il ne peut être affecté que par une adresse (d'où le &), plus précisément l'adresse d'un *char*.

a=4; // à son tour *a* est initialisé à une valeur de 4

fct(ptr); // équivalent à *fct(&a)*, le paramètre reçoit la valeur de *ptr*.

Remarque sur la dernière ligne :

fct(ptr) ; est un passage **par valeur de ptr**. Mais on peut aussi dire que c'est un passage **par référence de a**.

Les écritures *fct(ptr)* et *fct(&a)* ont le même effet. Dans les deux cas le paramètre prend pour valeur une adresse.

A l'intérieur de la fonction, on peut récupérer la valeur de *a* via le pointeur (c'est l'intérêt!) :

void fct (char param)*

{

int c;

char d;

c=param; // *c* contient la valeur de *param*, cad l'adresse de *a*

*d=*param;* // *d* contient la valeur de l'élément pointé, soit ici la valeur de *a*

}

2.2. Quand est on obligé d'utiliser un pointeur ?

→ tout simplement quand l'objet qu'on veut donner en paramètre à une fonction est trop gros !

Par exemple :

- **un tableau** (un string) : N octets à passer, cela ne peut se faire qu'en spécifiant l'adresse du premier octet,
- **une structure** : c'est un objet qui contient plusieurs variables en son sein, impossible de tous les passer à une fonction. Là encore c'est l'adresse de la structure qu'il faut passer (cad l'adresse de la première variable de la structure).

→ pour accéder aux registres de périphériques. La partie suivante donne le détail....

3. Structure, pointeur, et périphériques...

3.1. Pointeur sur structure ...

Déclarons un nouveau *type structuré* : que nous appelons *MyStrucType*...

```
typedef struct
{
    int a;
    char b;
}MyStrucType;
```

Définissons une variable de ce type :

```
MyStrucType MyVar;
char toto; // pour plus tard...
```

Comment accéder au champ **b** par exemple ? :

- en passant directement par *MyVar* :

toto = MyVar.b; // on utilise le .

- en passant par un pointeur sur structure :

*MyStructType *Ptr; // création du pointeur*

Ptr = &MyVar; // initialisation du pointeur

toto = Ptr → b; // accès à b via le pointeur. On remarque l'utilisation de la flèche.

3.2. Structures et périphériques ...

Un driver pour *STM32F103* (GPIO, TIMER, SPI ...) utilise **toujours** le petit fichier *header* nommé *stm32f10x.h* qui contient plus de 8000 lignes...

Parmi toutes les informations qu'il contient, on trouve des définitions de ***nouveaux types structurés***.

Chaque famille de périphériques (GPIO, TIMER, etc) est associé à un type particulier dont les champs portent le nom des registres qui composent le périphérique.

A titre d'exemple, prenons le *GPIO*. Un *<Ctrl-F>* dans le *.h* avec le mot clé *GPIO_TypeDef* amène directement au type recherché :

```
typedef struct
{
    __IO uint32_t CRL;
    __IO uint32_t CRH;
    __IO uint32_t IDR;
    __IO uint32_t ODR;
    __IO uint32_t BSRR;
    __IO uint32_t BRR;
    __IO uint32_t LCKR;
} GPIO_TypeDef;
```

0x4001 1800	
	Port D
0x4001 1400	
	Port C
0x4001 1000	
	Port B
0x4001 0C00	
	Port A
0x4001 0800	
	EXTI

Extrait du mapping mémoire du STM32F103

La figure de droite correspond à la position des registres du *GPIOA (Port A)* en *RAM*. On comprend que le *GPIOA* démarre à l'adresse 0x4001 0800 (adresse de CRL).

Pour atteindre les registres de *GPIOA*, il faut utiliser un pointeur sur le type *GPIO_TypeDef*, et surtout, il faut initialiser ce pointeur à l'adresse précise 0x4001 0800.

C'est précisément ce qui est fait pour nous, dans le fichier *.h*, mais de manière pour le moins originale.

Ainsi, ***GPIOA*** (défini dans *stm32f10x.h*) peut être vu comme un **pointeur sur un type *GPIO_TypeDef***, déjà initialisé à l'adresse **0x4001 0800**.

Donc à l'usage, nous pouvons écrire :

GPIOA → *ODR* pour atteindre directement la valeur du registre *ODR* (un champ de la structure) à partir de *GPIOA*.

4. Coup de pouce pour le premier driver **MyGPIO.h/.c**

Dans le monde de l'embarqué, les variables échangées dans les fonctions sont TRES souvent des structures (le cas des lib *LL*, *HAL* et autres...). C'est pourquoi nous reprenons cette manière de faire très utilisée notamment dans la fonction d'initialisation.

Voyons en détail le prototype exigé :

```
typedef struct
{
    GPIO_TypeDef * GPIO;
    char GPIO_Pin; // numéro de pin 0 à 15
    char GPIO_Conf; // see below
} MyGPIO_Struct_TypeDef;

#define In_Floating ...
#define In_PullDown
#define In_PullUp
#define In_Analog
#define Out_Ppull
#define Out_OD
#define AltOut_Ppull
#define AltOut_OD
```

4.1. Dans le programme principal :

On doit créer une variable de type *MyGPIO_Struct_TypeDef*, puis l'affecter selon ses besoins :

```
MyGPIO_Struct_TypeDef GPIO_Struct;  
GPIO_Struct.GPIO=GPIOB;  
GPIO_Struct.GPIO_Pin=12;  
GPIO_Struct.GPIO_Conf=In_PullDown;
```

Reste ensuite à appeler la fonction d'initialisation :

```
MyGPIO_Init(& GPIO_Struct);
```

Une fois exécutée, PB.12 est *input Pull Down*.

4.2. Dans la fonction MyGPIO_Init :

Il ne reste plus, dans la fonction, qu'à accéder aux divers registres du GPIO voulu puis à affecter les bonnes valeurs (CNF et MOD).

4.2.1. Accès à partir du paramètre de la fonction

```
Void MyGPIO_Init(MyGPIO_Struct_TypeDef* GPIOStructPtr)  
{  
    GPIOStructPtr->GPIO_Pin ... // pour accéder à la Pin  
    GPIOStructPtr->GPIO_Pin ... // pour accéder à la Conf  
    GPIOStructPtr->GPIO->ODR ... // pour accéder au registre ODR du GPIO considéré  
}
```

4.2.2. Accès par un pointeur intermédiaire pour accéder aux registres du GPIO

Si les deux flèches perturbent, on peut procéder en deux étapes :

```
Void MyGPIO_Init(MyGPIO_Struct_TypeDef* GPIOStructPtr)  
{  
    GPIO_TypeDef * MyGPIO; // on déclare un pointeur local sur un GPIO_TypeDef  
    MyGPIO= GPIOStructPtr -> GPIO; // Initialisation pointeur : MyGPIO devient ici égal à GPIOB (dans notre exemple)  
    MyGPIO -> ODR ... // accès classique à ODR...  
}
```